Lecture 7
Combinational Circuits - MSI Circuits

CSE260 : Digital Logic Design

Design Procedure

The design of combinational circuits starts from the verbal outline of the problem and ends in a logic circuit diagram

Design Procedure:

The procedure involves the following steps:

1. The problem is stated
2. Input and Output variables are determined
3. Letter symbols are assigned to the input and output variables
4. Truth table that defines relationship between input and outputs is derived.
5. The simplified boolean function for each output is obtained
6. The logic diagram is drawn

Hint: Aren't these the steps used for solving car-garage alarm system? So we are familiar with combinational circuit design!

Design Methods

- Different combinational circuit design methods:
  - Gate-level method (with logic gates)
  - Block-level design method
- Design methods make use of logic gates and useful functional blocks.
  - These are available as Integrated Circuit (IC) chips.

Design Methods

Why is it better to use IC than classical gate method?

- IC consists of gates all packed up together internally, which keeps the gates safe inside as well as makes it economical by reducing external interconnection wires
- As number of input increase, making the truth table and k-map gets cumbersome

Design Methods

- Type of IC chips (based on packing density) :

Small-scale integration (SSI)	up to 12 gates
Medium-scale integration (MSI)	12 - 99 gates
Large-scale integration (LSI)	100 - 9999 gates
Very large-scale integration (VLSI)	10,000 - 99,999 gates
Ultra large-scale integration (ULSI)	> 100,000 gates

Design Methods

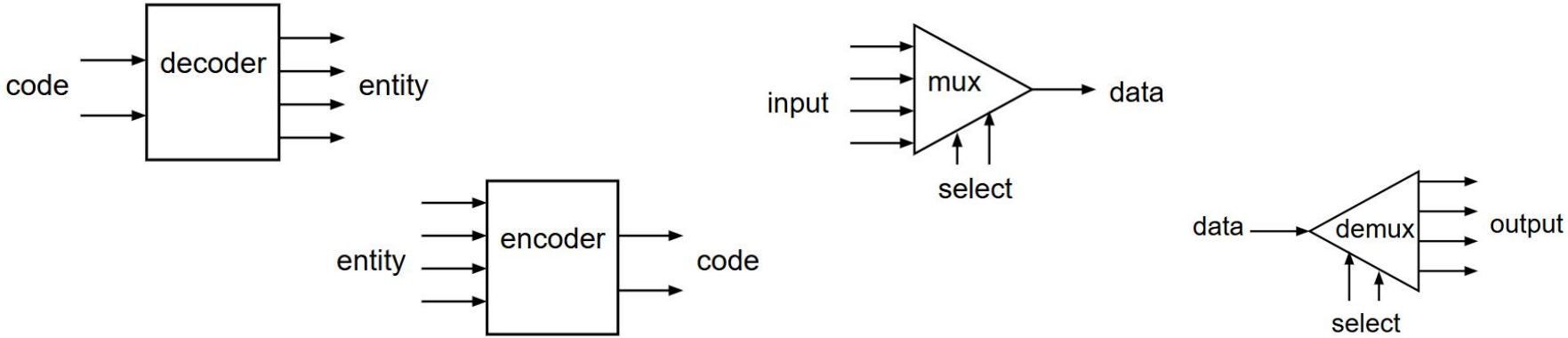
- *Main objectives of circuit design:*
  - (i) reduce cost
    - reduce number of gates (for SSI circuits)
    - reduce IC packages (for complex circuits)
  - (ii) increase speed
  - (iii) design simplicity (reuse blocks where possible)

Block-Level Design Method

- More complex circuits can be built using block-level method.
- In general, block-level design method (as opposed to gate-level design) relies on algorithms or formulae of the circuit, which are obtained by decomposing the main problem to sub-problems recursively (until small enough to be directly solved by blocks of circuits).

Useful MSI circuits

- Four common and useful MSI circuits are:
  - Decoder
  - Demultiplexer
  - Encoder
  - Multiplexer
- Block-level outlines of MSI circuits:



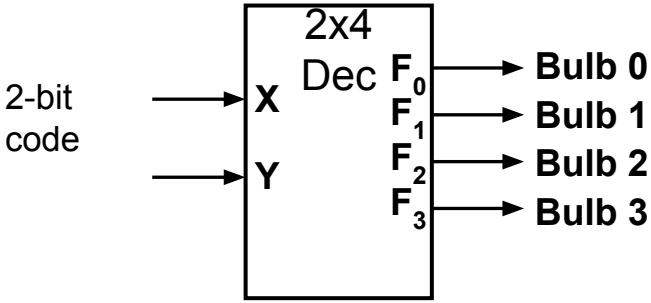
Decoders

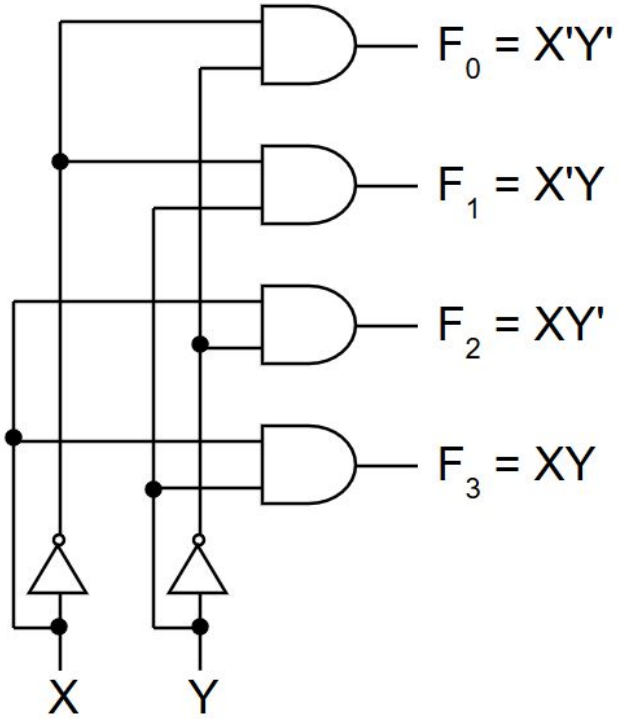
- Convert binary information from n input lines to (max. of) 2^n output lines.
- Known as n-to-m-line decoder, or simply n:m or $n \times m$ decoder ($m \leq 2^n$).
- May be used to generate 2^n (or fewer) minterms of n input variables.

Decoders

- From truth table, circuit for 2×4 decoder is:
- Note: Each output is a 2-variable minterm (**X′Y′**, **X′Y**, **XY′** or **XY**)

X	Y	F0	F1	F2	F3
0	0	**1**	**0**	**0**	**0**
0	1	**0**	**1**	**0**	**0**
1	0	**0**	**0**	**1**	**0**
1	1	**0**	**0**	**0**	**1**





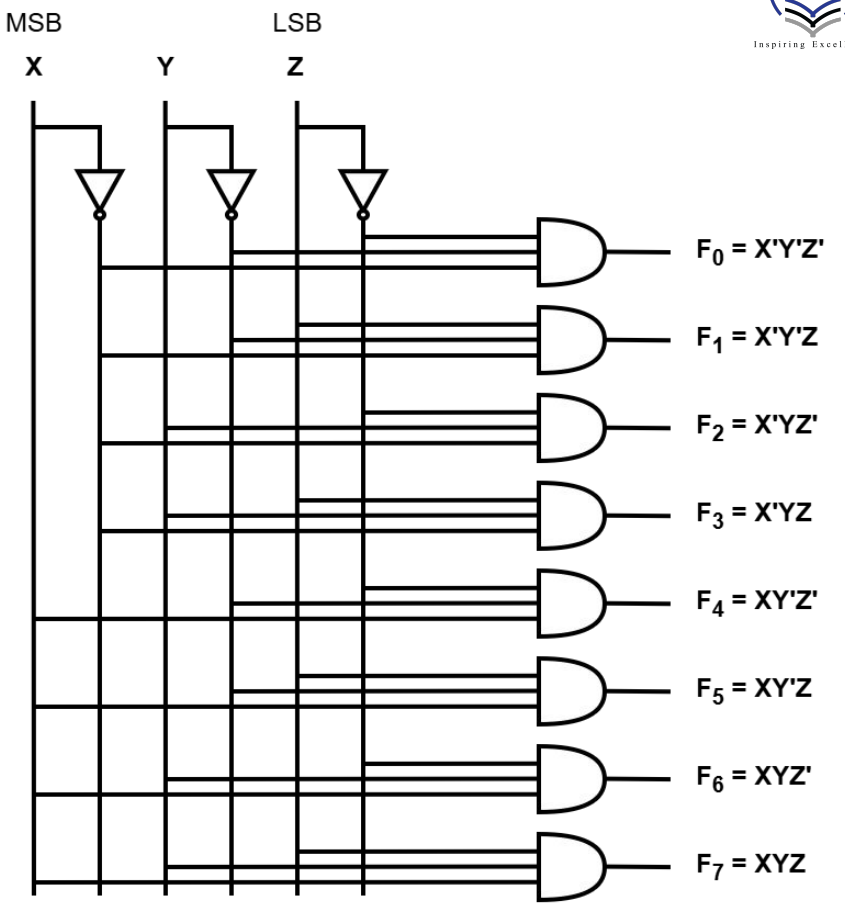
Decoders

Design a 3×8 decoder by yourself.

Decoders

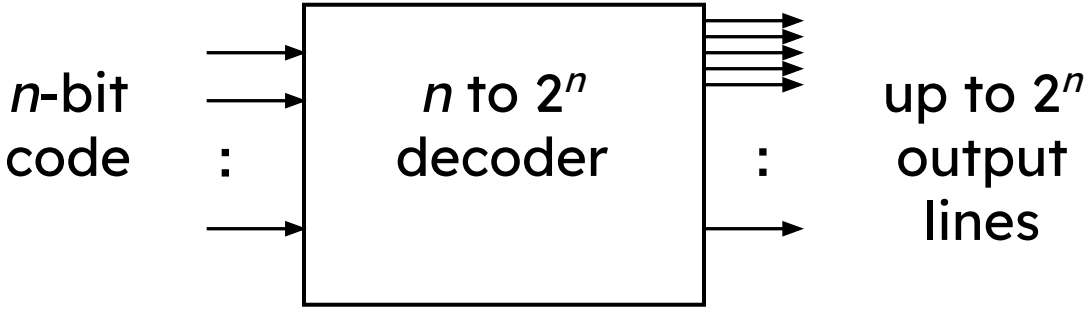
Solution:

x	y	z	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1



Decoders

In general, for an n-bit code, a decoder could select up to 2^n lines:



As n input generates 2^n output, which reminds us of canonical SOP, thus a decoder can be used to generate any function

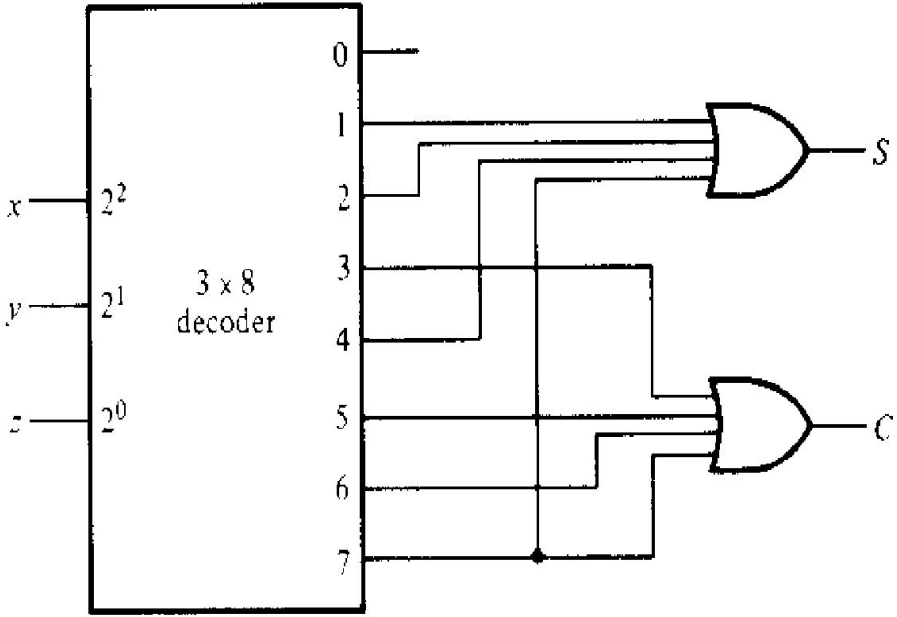
Decoders

Example 1: Full adder circuit with decoder (3 x 8 decoder)

We know,

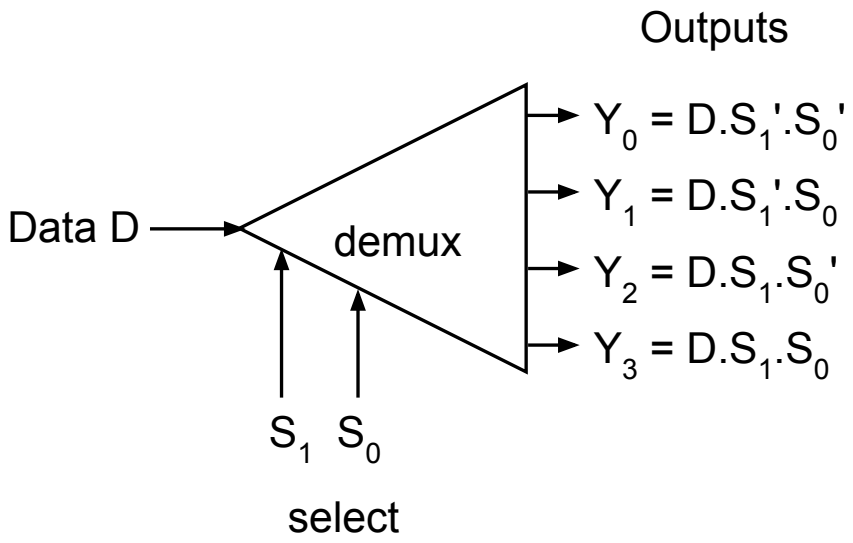
S(x,y,z) = Σ(1,2,4,7)

C(x,y,z) = Σ(3,5,6,7)



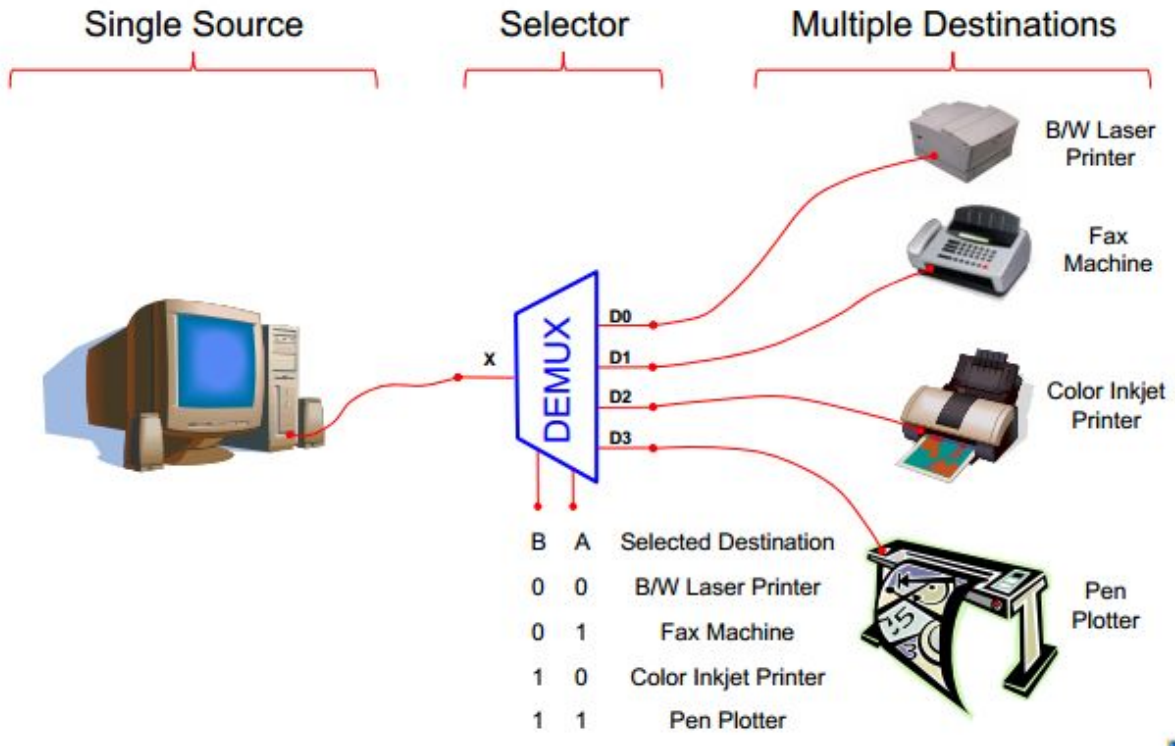
Demultiplexer

- Given an input line and a set of selection lines, the demultiplexer will direct data from input to a selected output line.
- An example of a 1-to-4 demultiplexer:



S_1	S_2	Y_0	Y_1	Y_2	Y_3
0	0	**D**	0	0	0
0	1	0	**D**	0	0
1	0	0	0	**D**	0
1	1	0	0	0	**D**

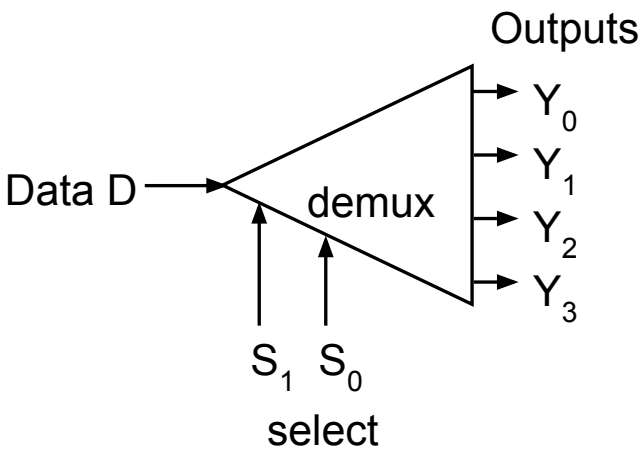
Typical Application of a DEMUX

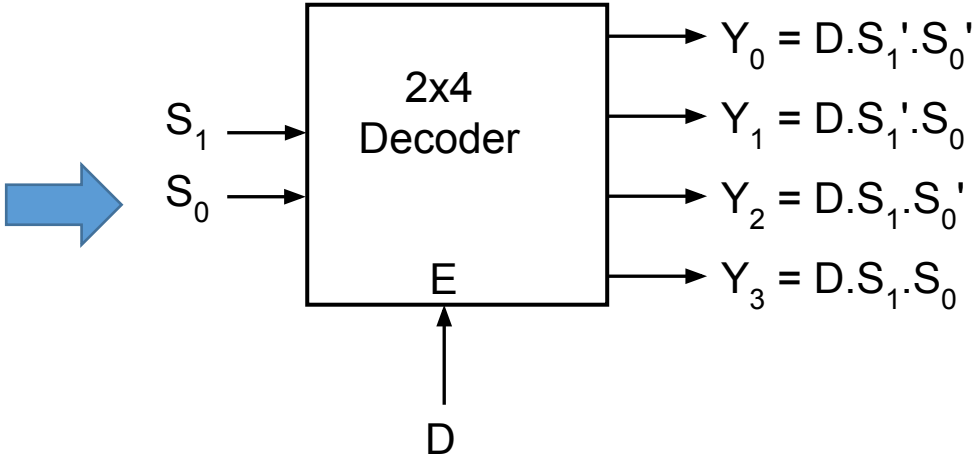


B	A	Selected Destination
0	0	B/W Laser Printer
0	1	Fax Machine
1	0	Color Inkjet Printer
1	1	Pen Plotter

Demultiplexer

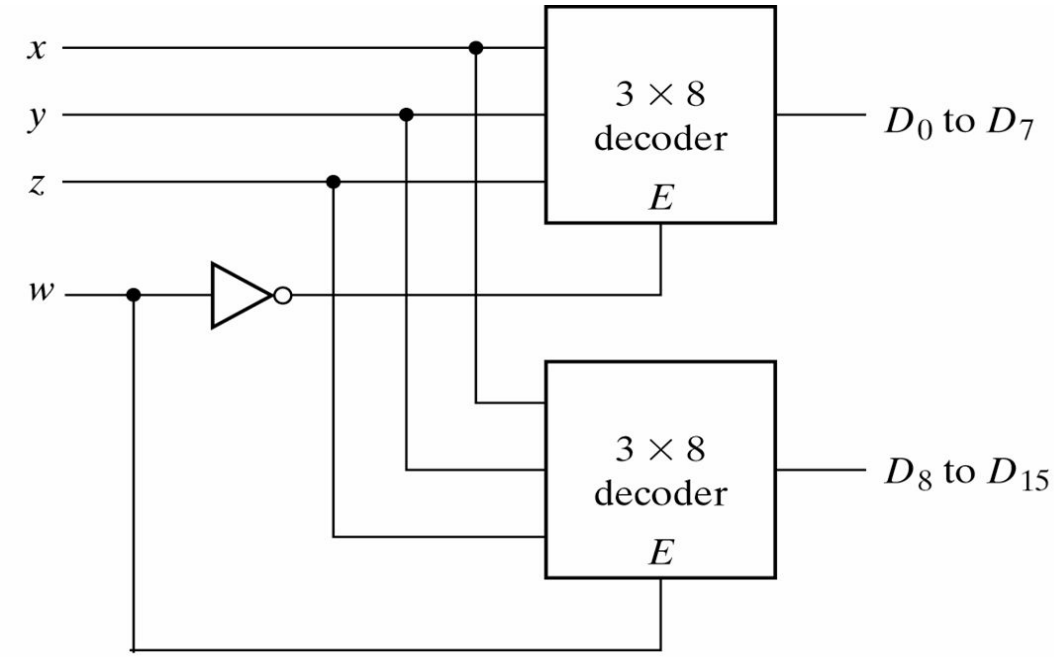
- The demultiplexer is actually identical to a decoder with enable. A decoder with an enable input can function as a demultiplexer (or demux)
  Decoder + enable= demultiplexer
- The selection of a specific output line is controlled by the bit values of ‘n’-selection lines.





Decoder (More Applications)

4-line-to-16 line Decoder constructed with two 3-line-to-8 line decoders with enables



Decoder

4-line-to-16 line Decoder constructed with two 3-line-to-8 line decoders with enables

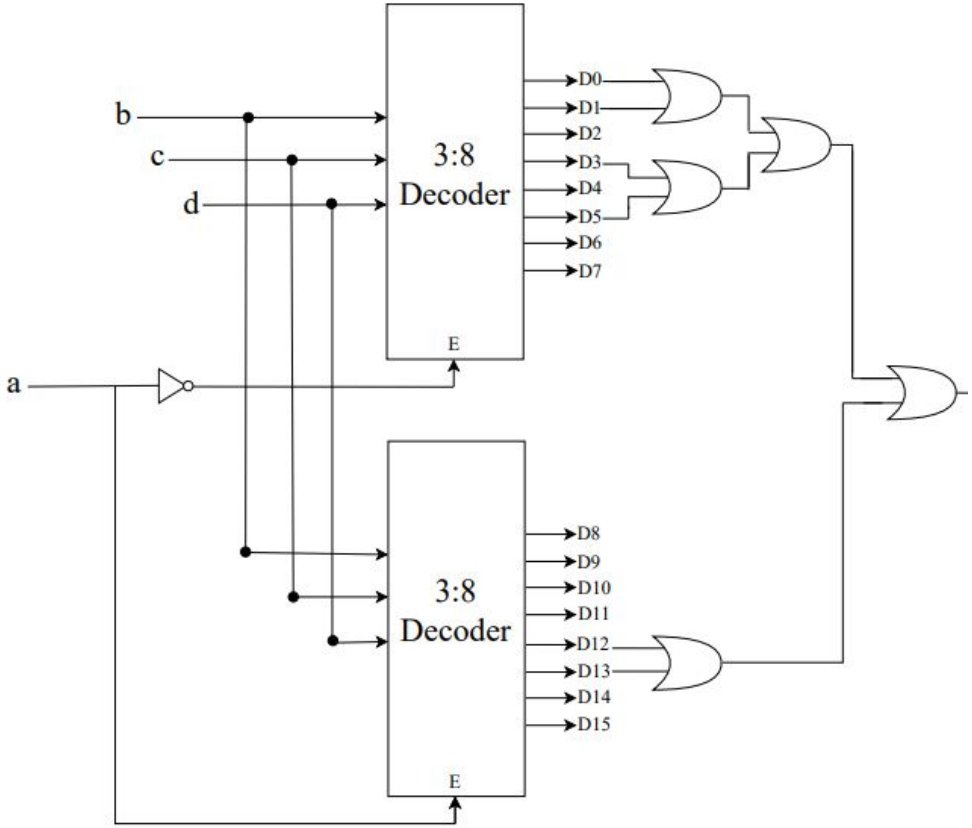
- When w=0, the top decoder is enabled and the other is disabled. The bottom decoder outputs are all 0's , and the top eight outputs generate min-terms 0000 to 0111.
- When w=1, the enable conditions are reversed. The bottom decoder outputs generate min-terms 1000 to 1111, while the outputs of the top decoder are all 0's.

W	X	Y	Z
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

Decoder

F(a,b,c,d)=Σ(0,1,3,5,12,13)
Implement the above boolean function using

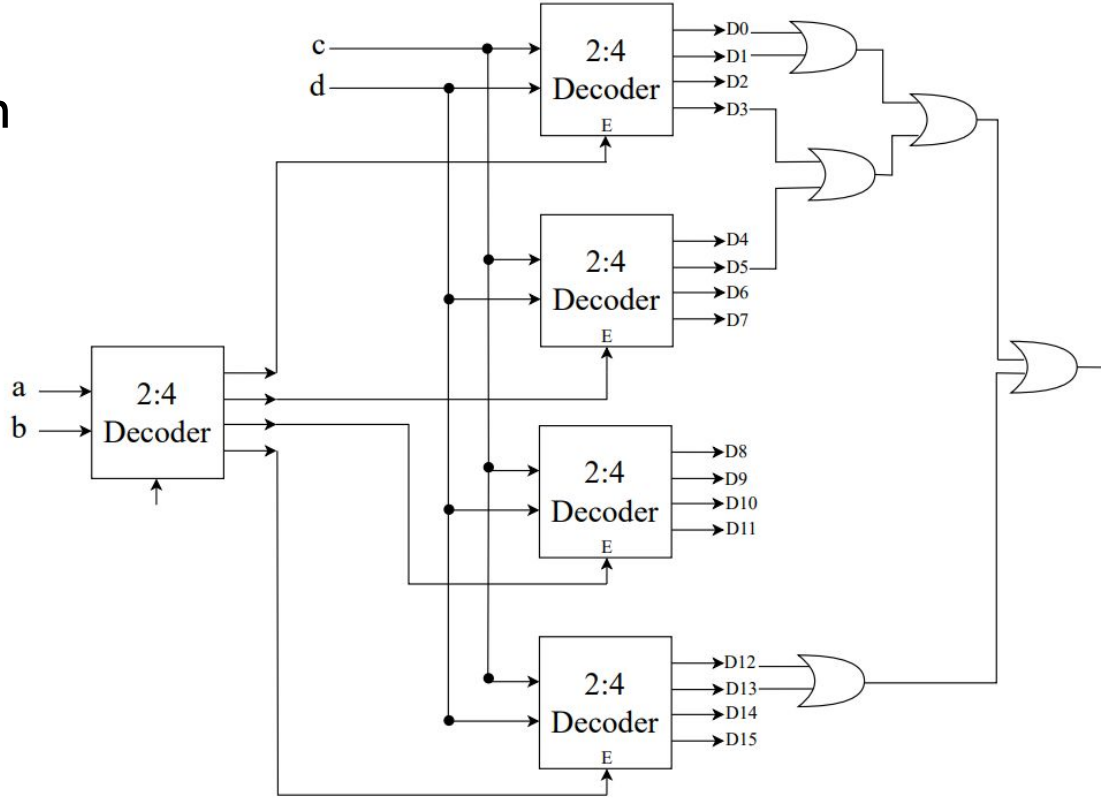
a. 3:8 Decoder(s).
b. 2:4 Decoder(s).



Decoder

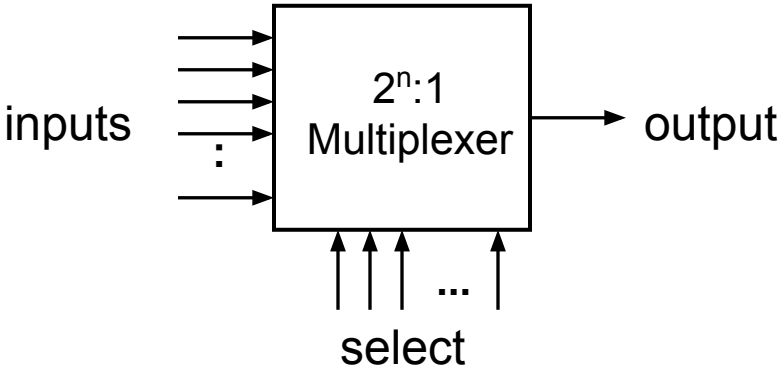
F(a,b,c,d)=Σ(0,1,3,5,12,13)
Implement the above boolean function using

a. 3:8 Decoder(s).
b. 2:4 Decoder(s).



Multiplexer

- A multiplexer is a device which has
  (i) a number of *input* lines
  (ii) a number of *selection* lines
  (iii) one *output* line
- A Multiplexer steers one of 2^n inputs to a single output line, using *n* selection lines. Also known as a *data selector.*

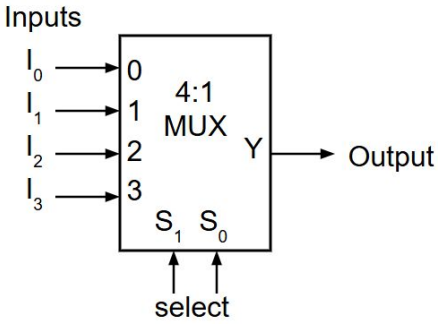


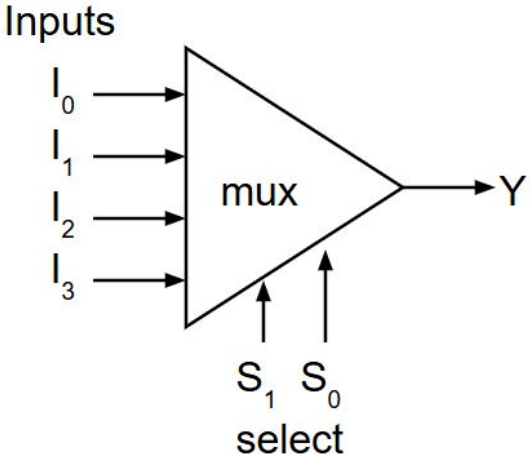
Multiplexer

Truth table for a 4-to-1 multiplexer:

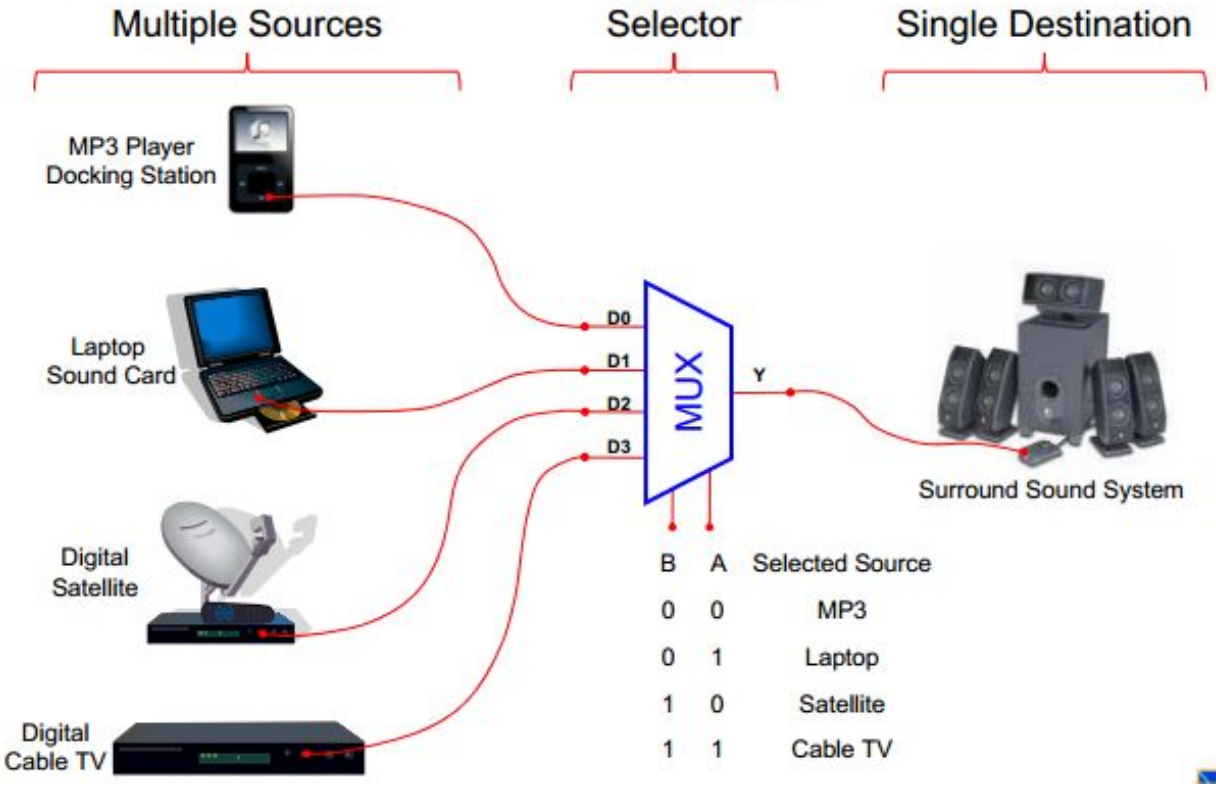
I_0	I_1	I_2	I_3	S_1	S_0	Y
d_0	d_1	d_2	d_3	0	0	d_0
d_0	d_1	d_2	d_3	0	1	d_1
d_0	d_1	d_2	d_3	1	0	d_2
d_0	d_1	d_2	d_3	1	1	d_3

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3





Typical Application of a MUX



B	A	Selected Source
0	0	MP3
0	1	Laptop
1	0	Satellite
1	1	Cable TV

Multiplexer

- Output of multiplexer is

  "sum of the (product of *data lines* and *selection lines*)"

- Often known as Data selector as it selects one of the many inputs and steers the binary information to the output line.

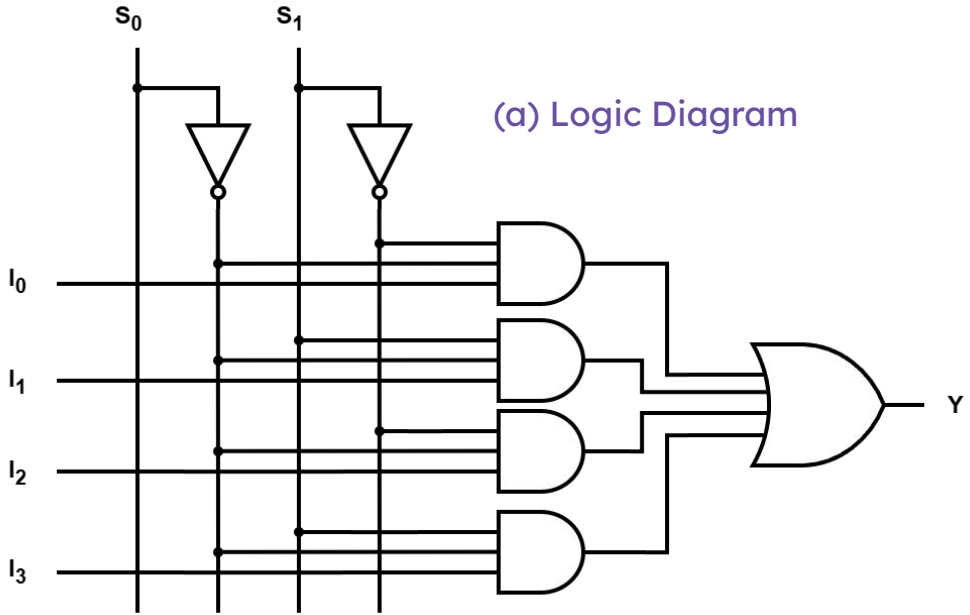
- Example: the output of a 4-to-1 multiplexer is:

$$Y = I_0.(S_1'.S_0') + I_1.(S_1'.S_0) + I_2.(S_1.S_0') + I_3.(S_1.S_0)$$

Try it yourself

Draw the internal circuit diagram (logic diagram) of a 4-to-1 multiplexer.

Solution: $Y = I_0.(S_1'.S_0') + I_1.(S_1'.S_0) + I_2.(S_1.S_0') + I_3.(S_1.S_0)$

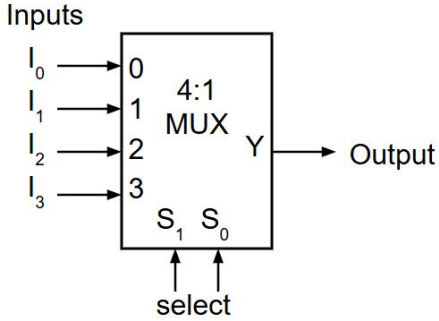


(a) Logic Diagram

(b) Function Table

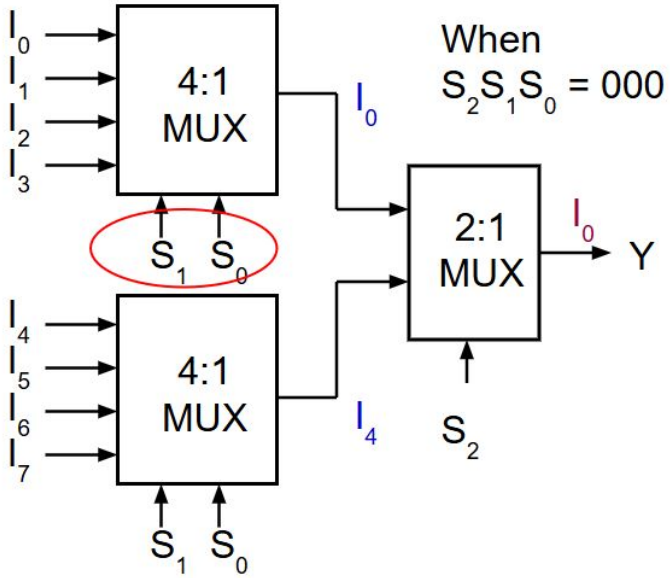
S_1	S_0	Y
0	0	d_0
0	1	d_1
1	0	d_2
1	1	d_3

(c) Block Diagram



Larger Multiplexers

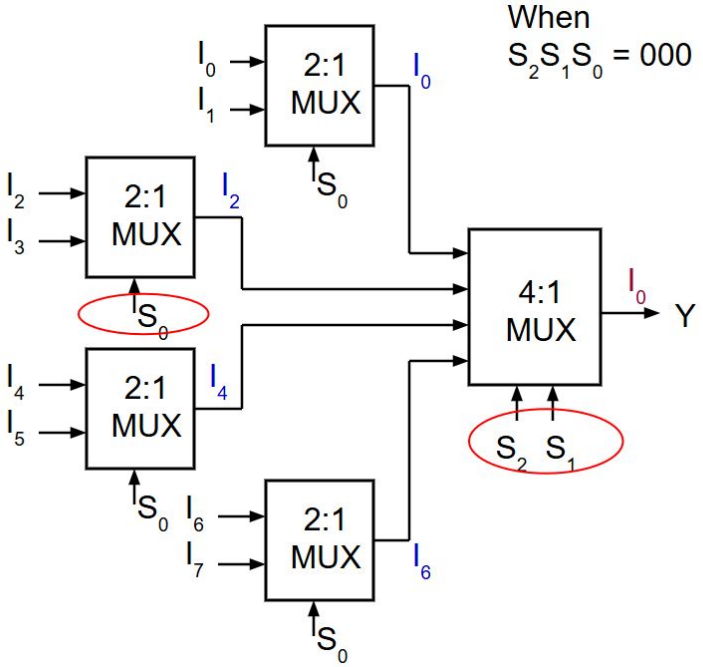
- Larger multiplexers can be constructed from smaller ones.
- An 8-to-1 multiplexer can be constructed from smaller multiplexers like this (from two 4x1 and one 2x1):



S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

Larger Multiplexers

- Another implementation of an 8-to-1 multiplexer using smaller multiplexers (four 2x1 and one 4x1):



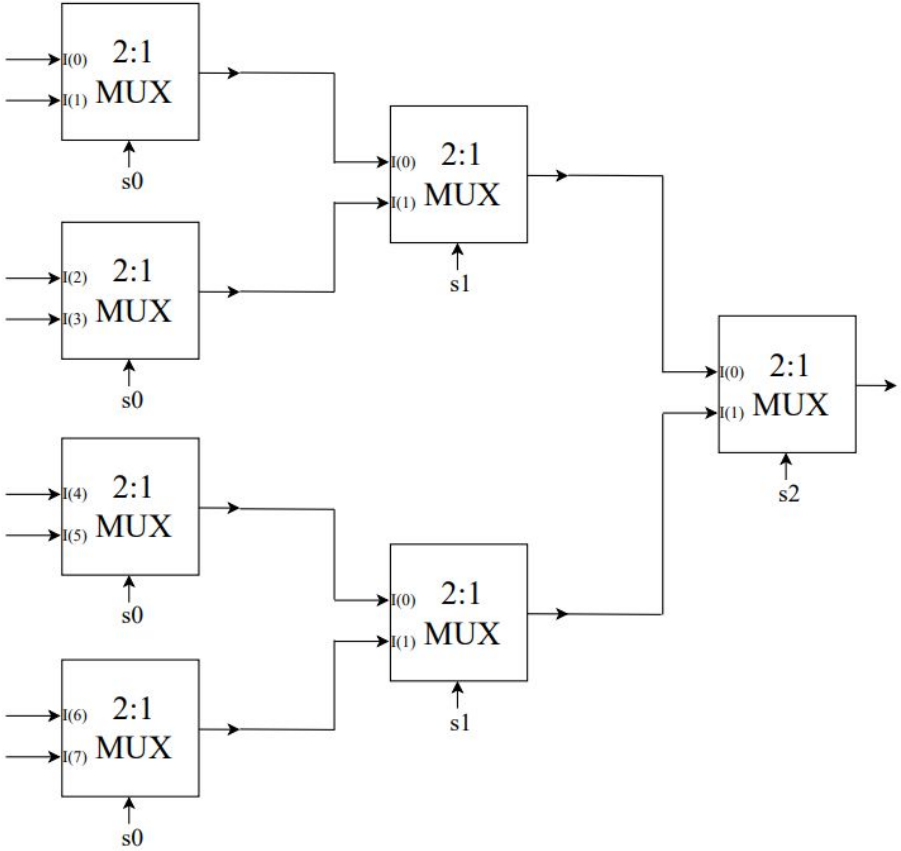
S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

Q: Can we use only 2:1 multiplexers?

Larger Multiplexers

Q: Can we use only

2:1 multiplexers?



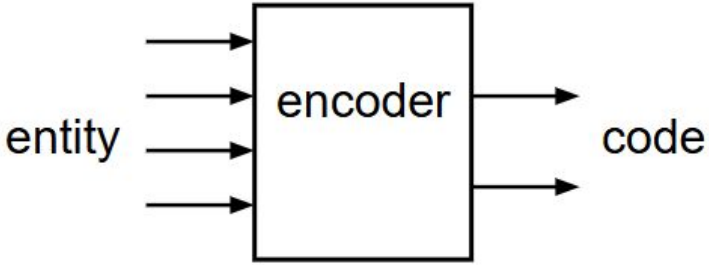
Try it yourself: Larger Multiplexers

Q: A 16-to-1 multiplexer can be constructed from only 4-to-1 multiplexers:

Encoder

Encoder is a digital function that produces a reverse operation of a decoder!

It has 2^n input lines and n output lines



Example: Octal-binary encoder

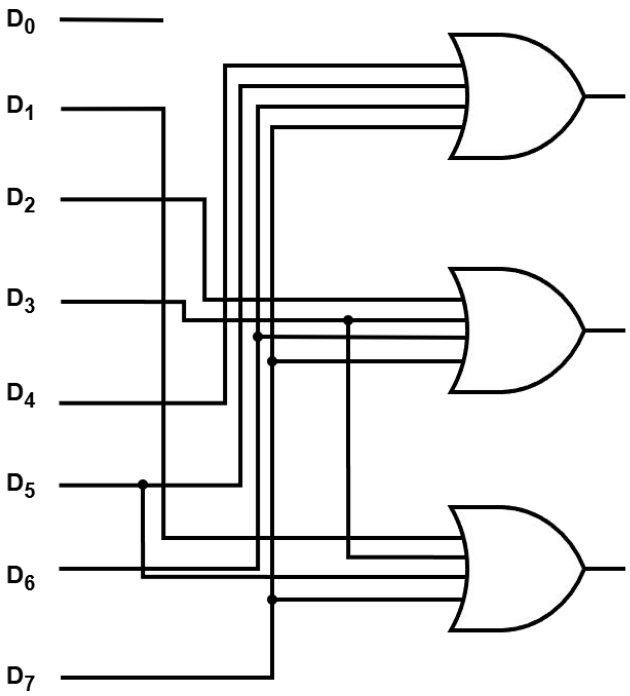
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

$$X = D_4 + D_5 + D_6 + D_7$$

$$Y = D_2 + D_3 + D_6 + D_7$$

$$Z = D_1 + D_3 + D_5 + D_7$$

Example: Octal-binary encoder



$$X = D_4 + D_5 + D_6 + D_7$$

$$Y = D_2 + D_3 + D_6 + D_7$$

$$Z = D_1 + D_3 + D_5 + D_7$$

Octal to Binary Encoder Logic Diagram

There are 8 input variables so there will be 256 input combination, amongst which in Octal-binary encoder only 8 are useful

Priority Encoder

- Accepts multiple values and encodes them
  - Works when more than one input is active
- Consists of:
  - Inputs (2^n)
  - Outputs
    - when more than one output is active, sets output to correspond to highest input
    - V (indicates whether any of the inputs are active). This helps to show the output when all inputs are 0s
  - Selectors / Enable

Example: Priority Encoder

- Design a priority encoder, which will allow more than one input to exist and encodes only the highest priority input line. For example, 8x3 encoder in a if user give D2 and D7 together, it will allow data of D7 to pass.

Example: 4x2 Priority Encoder

- Design a priority encoder, which will allow more than one input to exist and encodes only the highest priority input line. For example, 4x2 encoder in a if user give D2 and D3 together, it will allow data of D3 to pass.

Example: 4x2 Priority Encoder

Designing a 4x2 priority encoder

D_3	D_2	D_1	D_0	A_1	A_0	V
0	0	0	0	**X**	**X**	0
0	0	0	1	**0**	**0**	1
0	0	1	0	**0**	**1**	1
0	0	1	1	**0**	**1**	1
0	1	0	0	**1**	**0**	1
0	1	0	1	**1**	**0**	1
0	1	1	0	**1**	**0**	1
0	1	1	1	**1**	**0**	1
1	0	0	0	**1**	**1**	1
1	0	0	1	**1**	**1**	1
1	0	1	0	**1**	**1**	1
1	0	1	1	**1**	**1**	1
1	1	0	0	**1**	**1**	1
1	1	0	1	**1**	**1**	1
1	1	1	0	**1**	**1**	1
1	1	1	1	**1**	**1**	1

Note: Amongst all 3 input, D_2 is highest so output reflects the binary value of 2

Note: V is not really an output, just shows whether there is an active input or not.

From the truth table

$A_1 = \Sigma(4,5,6,7,8,9,10,11,12,13,14,15) + d(0)$

$A_0 = \Sigma(2,3,8,9,10,11,12,13,14,15) + d(0)$

Example: 4x2 Priority Encoder

$A_1 = \Sigma(4,5,6,7,8,9,10,11,12,13,14,15) + d(0)$

$A_0 = \Sigma(2,3,8,9,10,11,12,13,14,15) + d(0)$

D_3D_2 \ D_1D_0	00	01	11	10
00	X			
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

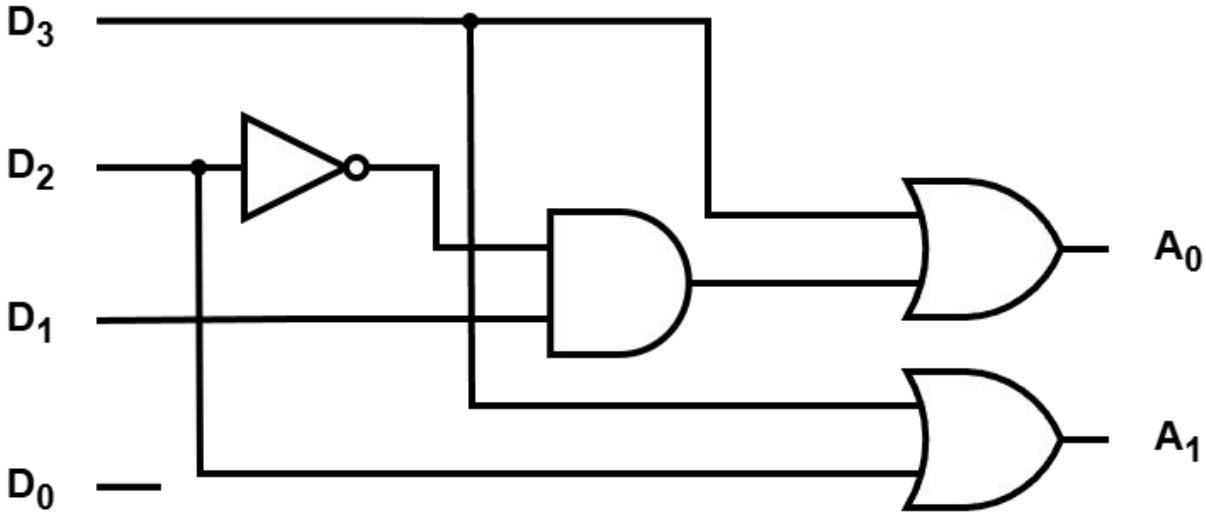
$A_1 = D_2 + D_3$

D_3D_2 \ D_1D_0	00	01	11	10
00	X		1	1
01				
11	1	1	1	1
10	1	1	1	1

$A_0 = D_3 + D_1D_2'$

Example: 4x2 Priority Encoder

- $A_1 = D_2 + D_3$ $\quad\quad$ $A_0 = D_3 + D_1D_2'$

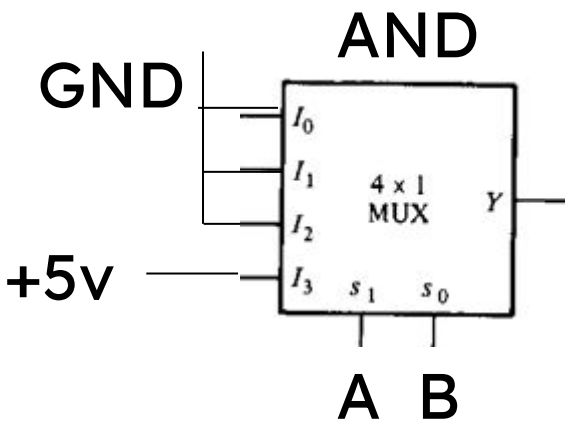


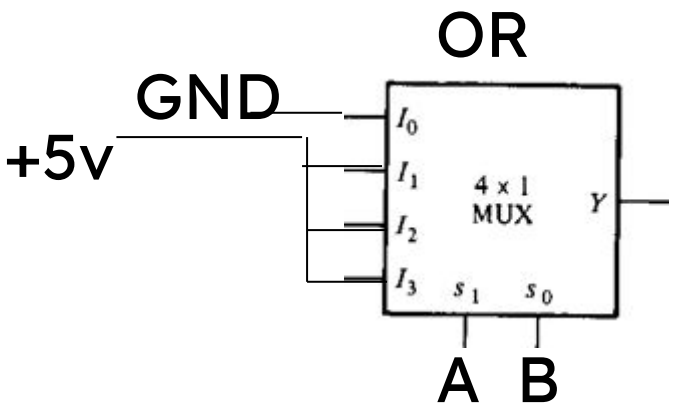
Boolean Function Implementation using MSI Circuits

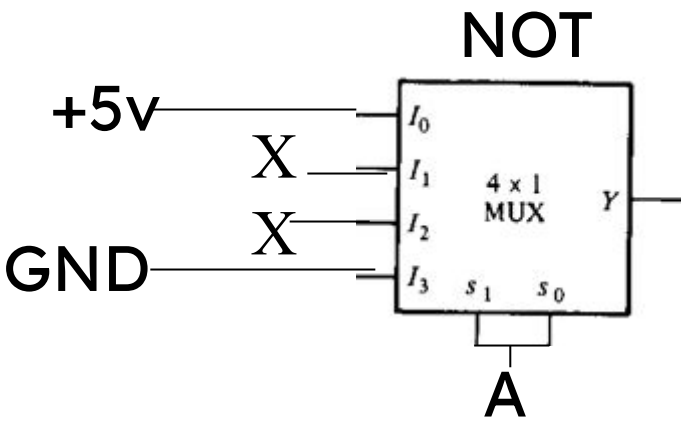
Try it yourself

Using 4:1 MUX design

- AND gate
- OR gate
- NOT gate







Try it yourself

a)Build the following function using 8x1 Mux.

$F(A,B,C)=\Sigma(0,4,5)$

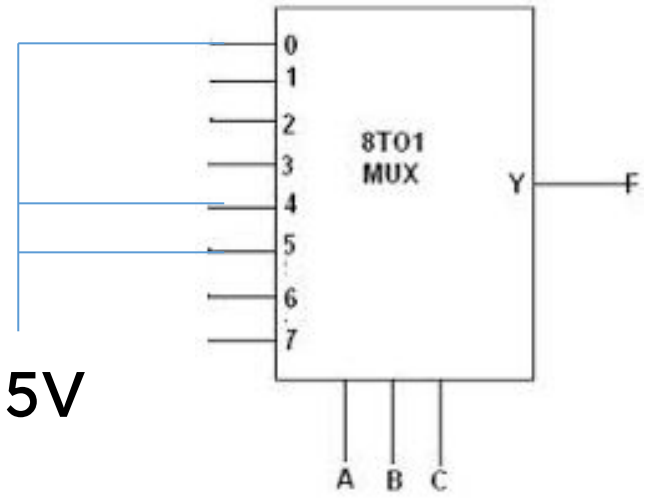
b)Design same thing with 3x8 decoder

c) Try designing it with single 4x1 Mux

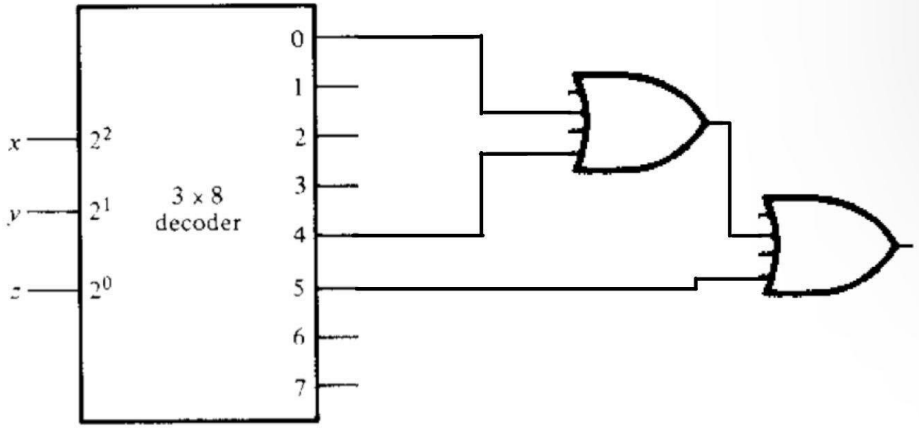
Solution

F=Σ(0,4,5)

a)



b)



Rules: for using smaller mux to build larger equation

If the two minterms in a column are not circled, apply 0 to the corresponding multiplexer input.

If the two minterms are circled, apply 1 to the corresponding multiplexer input.

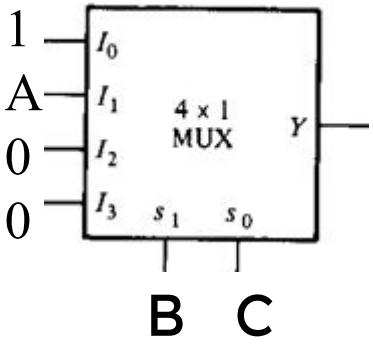
If the bottom minterm is circled and the top is not circled, apply A to the corresponding multiplexer input.

If the top minterm is circled and the bottom is not circled, apply A′ to the corresponding multiplexer input.

Solution

Answer: Part (c)

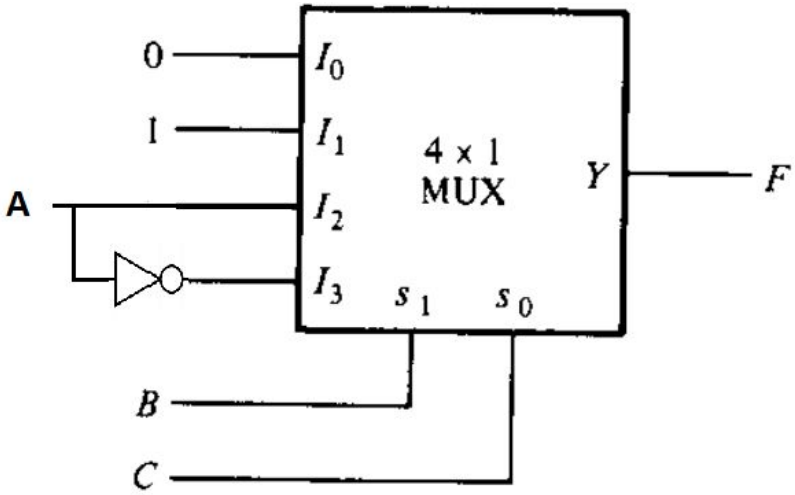
	I_0	I_1	I_2	I_3
A'	0	1	2	3
A	4	5	6	7
	1	A	0	0



Build the following function using 4x1 MUX

$F(A,B,C) = \Sigma\ (1, 3, 5, 6)$

	I_0	I_1	I_2	I_3
A'	0	1	2	3
A	4	5	6	7
	0	1	A	A'



Try it yourself

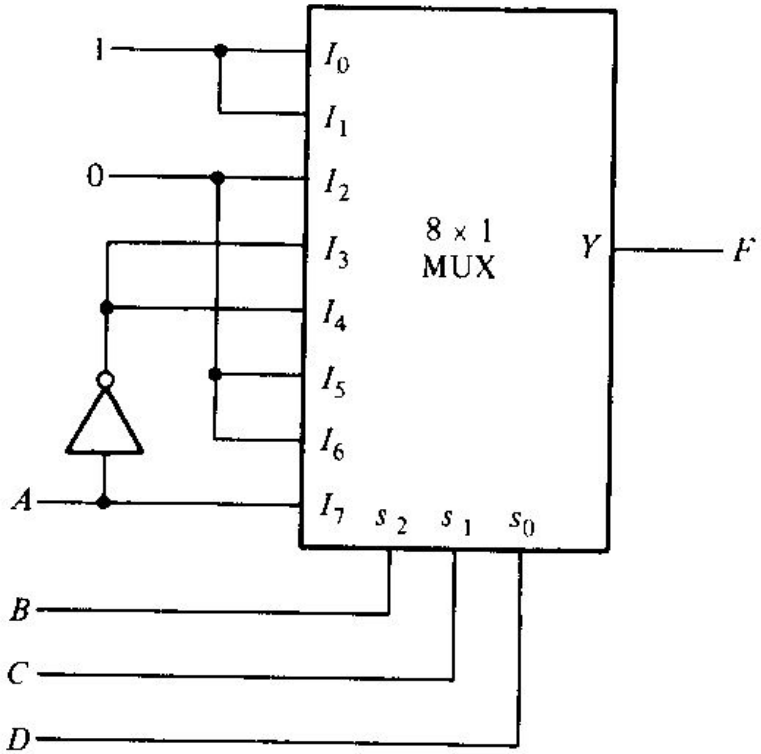
a. Implement the below function using a 8x1 Mux.
b. Implement the below function using a 4x16 decoder and OR gates

$F(A,B,C,D) = \Sigma (0,1,3,4,8,9,15)$

Solution

a.

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
A'	0	1	2	3	4	5	6	7
A	8	9	10	11	12	13	14	15
	1	1	0	A'	A'	0	0	A



Home-task: Try it yourself

Using 2:1 MUX design

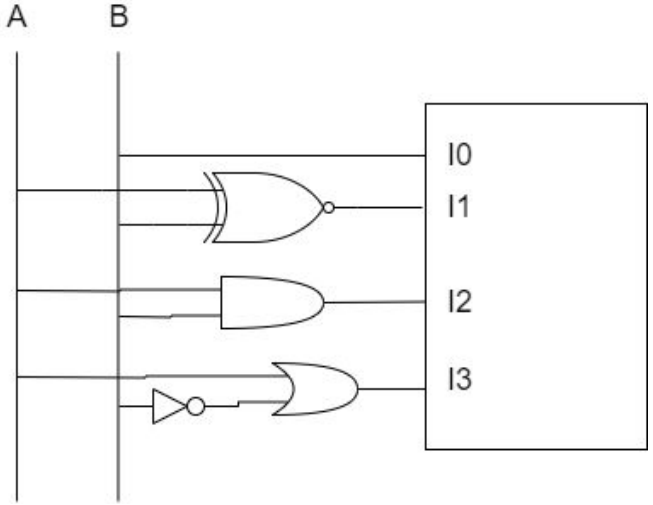
- AND gate
- OR gate
- NOT gate

- Solution: Try YOUTUBE

4 variables with 4x1 Mux

$F(A,B,C,D) = \Sigma\ (1,3,4,11,12,13,14,15)$

	I_0	I_1	I_2	I_3
A'B'	0	1	2	3
A'B	4	5	6	7
AB'	8	9	10	11
AB	12	13	14	15
	B	A⊙B	AB	A+B'



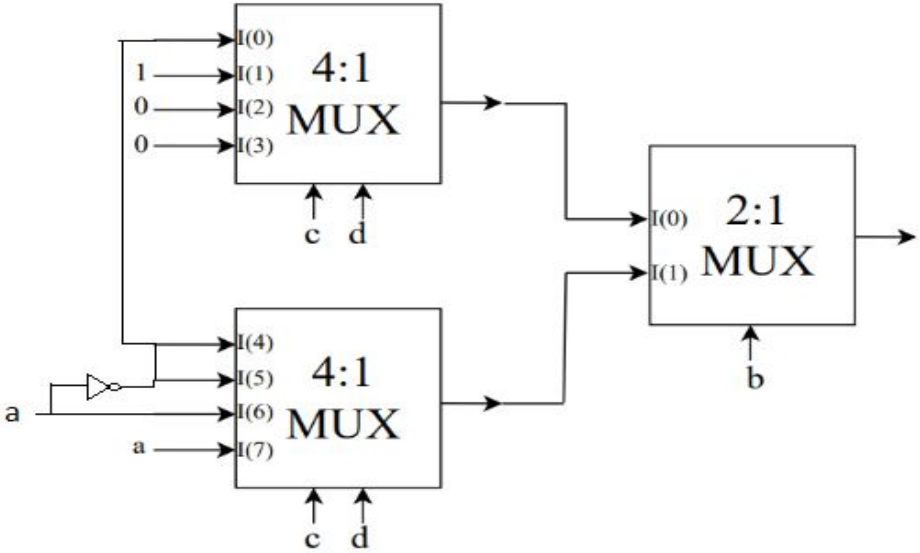
A	B	C	D	min
0	0	0	0	**0**
0	0	0	1	**1**
0	0	1	0	**2**
0	0	1	1	**3**
0	1	0	0	**4**
0	1	0	1	**5**
0	1	1	0	**6**
0	1	1	1	**7**
1	0	0	0	**8**
1	0	0	1	**9**
1	0	1	0	**10**
1	0	1	1	**11**
1	1	0	0	**12**
1	1	0	1	**13**
1	1	1	0	**14**
1	1	1	1	**15**

https://www.youtube.com/watch?v=c8lhlAX2JMk

Try it yourself

Implement the boolean function using two 4:1 MUX(s) and one 2:1 MUX.

F(A,B,C,D) = Σ (0,1,4,5,9,14,15)



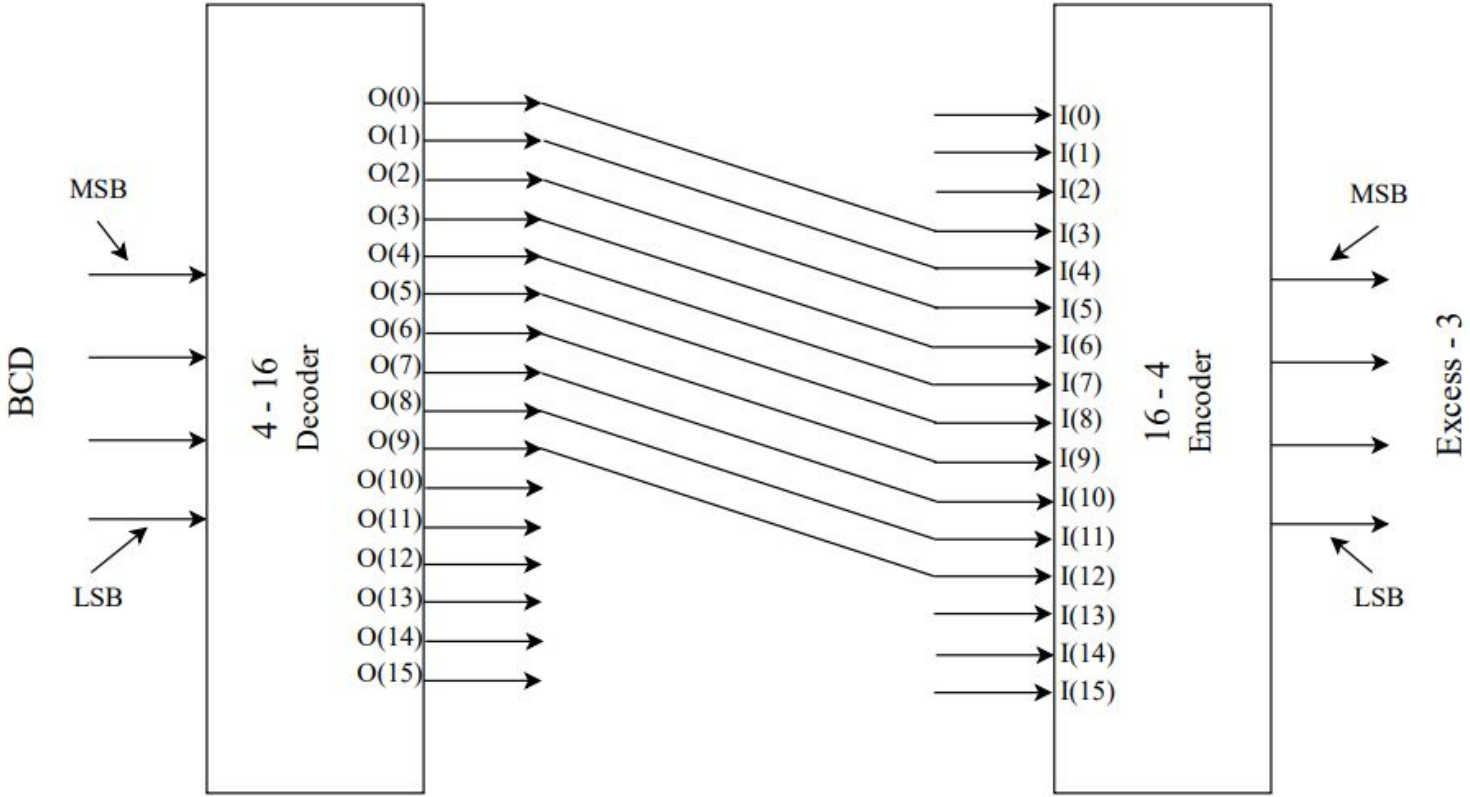
	I(0)	I(1)	I(2)	I(3)	I(4)	I(5)	I(6)	I(7)
a'	0	1	2	3	4	5	6	7
a	8	9	10	11	12	13	14	15
	a'	1	0	0	a'	a'	a	a

Combining MSI to build Combinational Design Circuit

Exercise time!

Design a BCD to Excess 3 code converter using ‘4x16’ decoder and ‘16x4’ encoder

Solution:

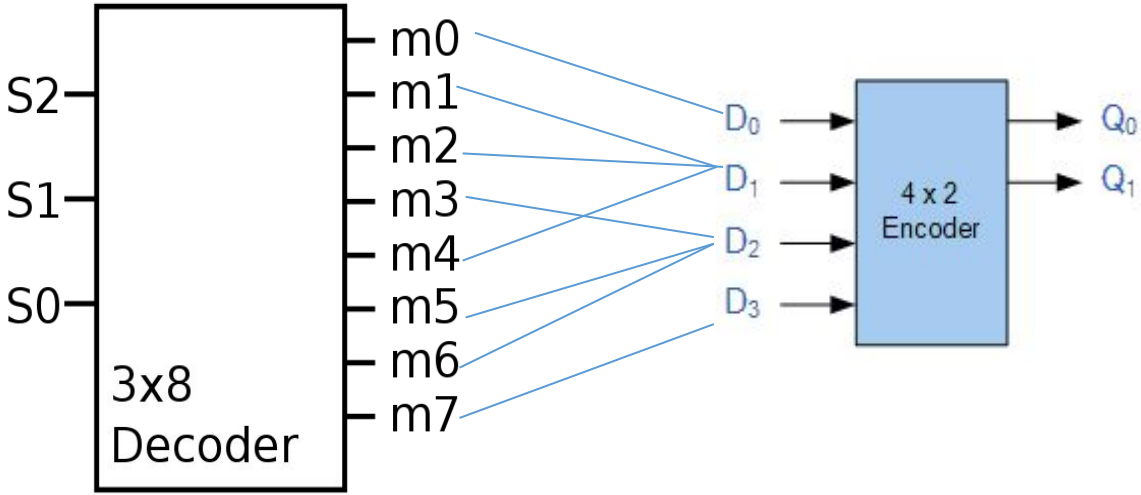

BCD
MSB
LSB
4 - 16
Decoder
O(0)
O(1)
O(2)
O(3)
O(4)
O(5)
O(6)
O(7)
O(8)
O(9)
O(10)
O(11)
O(12)
O(13)
O(14)
O(15)
I(0)
I(1)
I(2)
I(3)
I(4)
I(5)
I(6)
I(7)
I(8)
I(9)
I(10)
I(11)
I(12)
I(13)
I(14)
I(15)
16 - 4
Encoder
MSB
LSB
Excess - 3

Exercise time!

Design a full adder using ‘3x8’ decoder and ‘4x2’ encoder

Solution

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



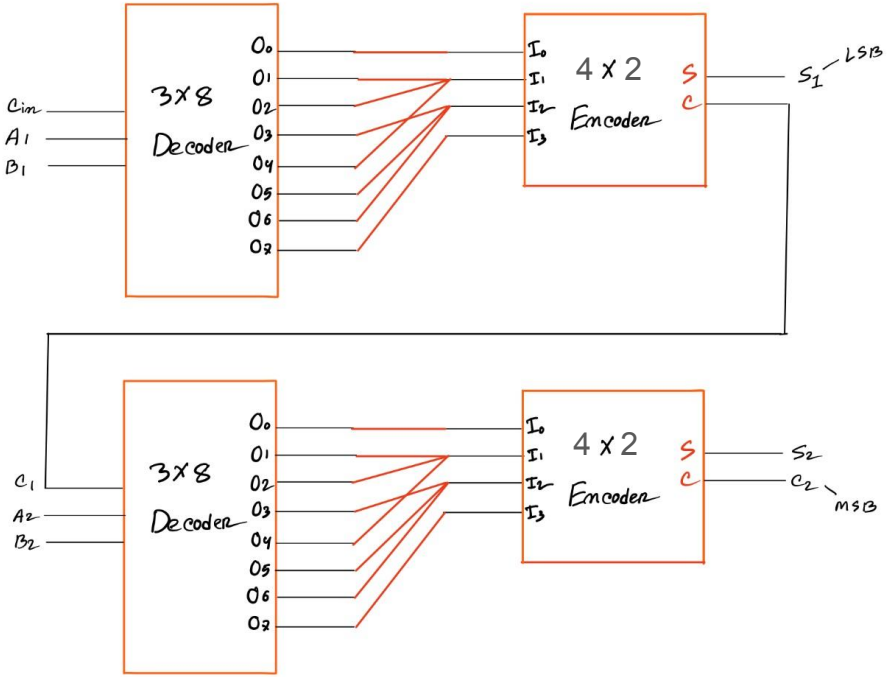
Exercise time!

Design a 2-bit parallel adder where you have to use appropriate encoders and decoders to build the internal circuit.

Solution

$$A = \overset{MSB}{A_2}\ \overset{LSB}{A_1}$$

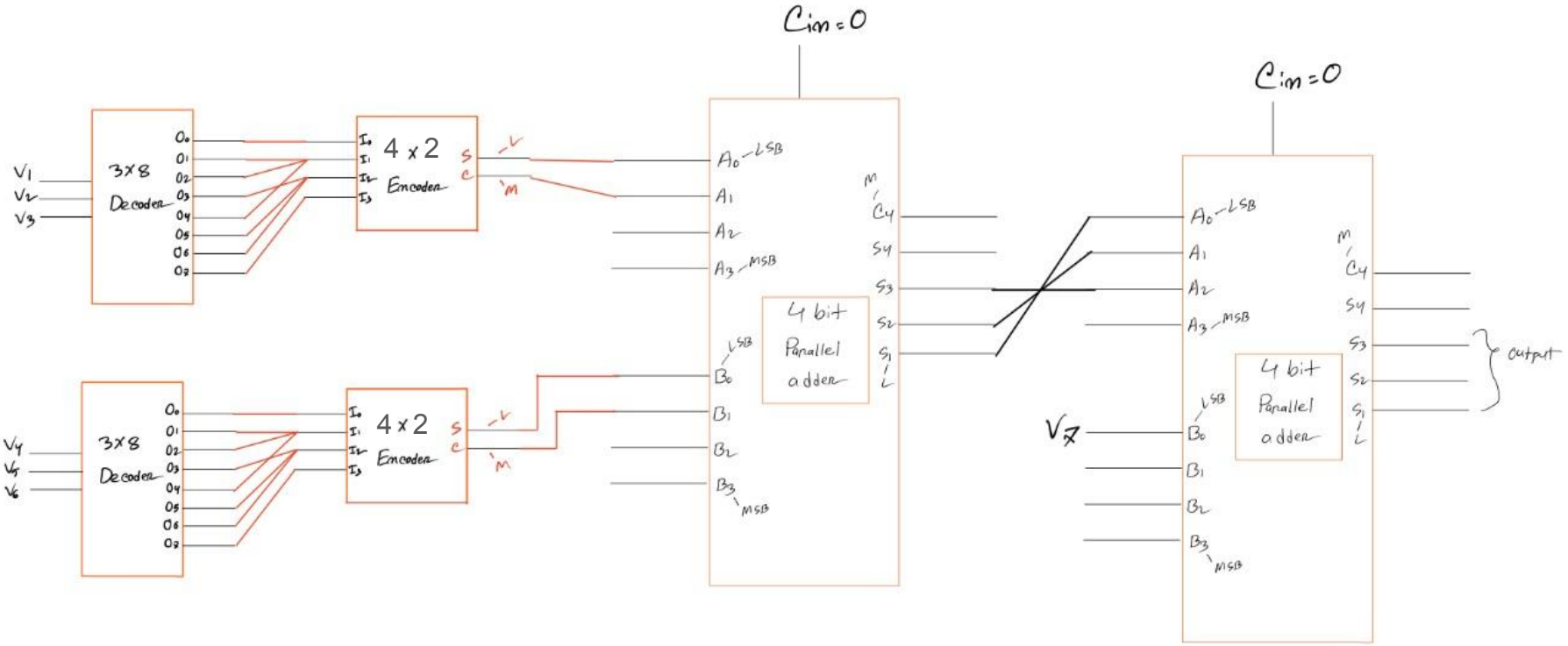
$$B = \underset{MSB}{B_2}\ \underset{LSB}{B_1}$$



Exercise time!

Design a 7 person voting system using encoder(s) and decoder(s) and parallel adder(s). You can not use full adders. Meaning, for the inputs, you MUST use encoder(s) and decoder(s) as required.

Solution


Cin = 0
Cin = 0
V1
V2
V3
3x8
Decoder
4 x 2
Encoder
S
C
L
M
A0 LSB
A1
A2
A3 MSB
B0 LSB
B1
B2
B3 MSB
4 bit
Parallel
adder
C4
S4
S3
S2
S1
V4
V5
V6
V7
output

Try Yourself!

Design a 19 person voting system using encoder(s) and decoder(s) and parallel adder(s). You can not use full adders. Meaning, for the inputs, you MUST use encoder(s) and decoder(s) as required. Make sure your circuit is optimized.